Foundations

DuckDB, dbt & How They Fit Together

What this handout covers

You've met the tools (session 1) and built your first model (session 2). Before we layer on sources, seeds and tests in session 3, a quick **consolidation** of the pieces and how they fit together:

1. The **data warehouse** (DuckDB, in this course)
2. The **dbt** framework
3. The **files** that make up a dbt project — and how they connect to the warehouse

By the end you should be able to point at any file in the repo and say what job it does.

Database vs Data Warehouse

Both are systems for storing structured data with SQL access. The difference is what they're **optimized for**.

	Database (OLTP)	Data Warehouse (OLAP)
Job	Run the app	Answer analytical questions
Workload	Many small reads/writes	Big scans & aggregations
Rows per query	1s–1,000s	Millions
Examples	Postgres, MySQL	Snowflake, BigQuery, DuckDB

In analytics engineering, we almost always work with a **warehouse**.

DuckDB — our warehouse

Think "**SQLite for analytics**":

- **In-process**: no server. The database is a single file (`my_database.duckdb`).
- **Columnar & vectorized**: fast on the aggregation workloads a warehouse sees.
- **Reads Parquet/CSV directly**: no ingestion step needed for many files.
- **Free, open-source, cross-platform**.

We use DuckDB because it has the *shape* of a real warehouse (Snowflake, BigQuery) but runs entirely on your laptop — perfect for learning.

What dbt is (and isn't)

dbt is a framework for transforming data *inside* a warehouse using SQL.

What dbt IS	What dbt is NOT
A SQL compiler + runner	A database or warehouse
A dependency & testing framework	An ETL/ingestion tool
A documentation generator	A scheduler (out of the box)
Git-versioned transformation code	A BI or dashboarding tool

You bring the warehouse; dbt orchestrates the SQL you run against it.

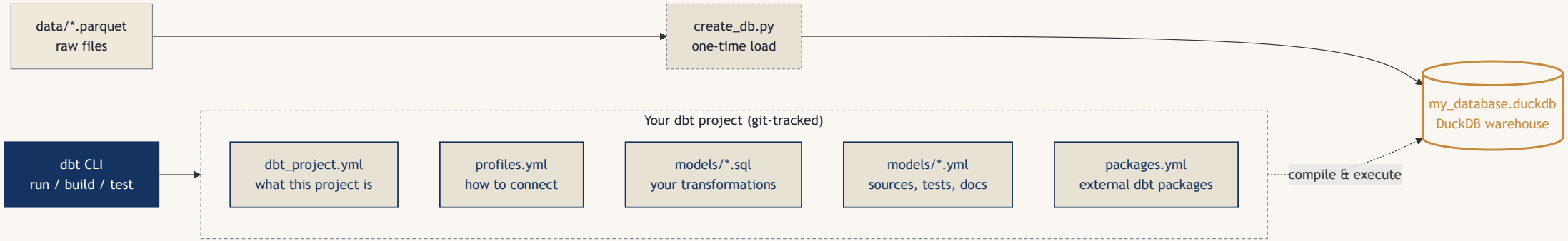
dbt Core (what we use) is the open-source CLI. **dbt Cloud** adds a hosted UI, scheduler, and IDE.

The main dbt files

File / folder	Role
`dbt_project.yml`	Project config — name, paths, default materializations
`profiles.yml`	Connection info — adapter, database path, schema, target
`packages.yml`	External dbt packages (e.g. `dbt_utils`)
`models/*.sql`	Your transformations (one `SELECT` per file)
`models/*.yml`	Metadata: sources, tests, descriptions, contracts
`seeds/*.csv`	Small static reference data, loaded as tables

`snapshots/` and `macros/` come later in the course.

Putting it all together



1. `create_db.py` loads Parquet files into the DuckDB warehouse (one-time).
2. `dbt` reads the project (configs + SQL + YAML) and compiles Jinja → pure SQL.
3. Using `profiles.yml`, dbt connects to DuckDB and **executes** the SQL, creating tables and views.

A `dbt run` in motion

```
1. dbt reads   → dbt_project.yml + profiles.yml + models/
2. dbt parses  → builds the DAG from ref() and source() calls
3. dbt compiles → Jinja templates → pure SQL (stored in target/compiled/)
4. dbt connects → opens my_database.duckdb via profiles.yml
5. dbt executes → runs each model in DAG order, materializing tables/views
6. dbt logs    → writes run_results.json + manifest.json to target/
```

Everything in `target/` is *disposable* — delete it any time and re-run.

What's next

- **Session 3** — Sources, seeds, and first tests. You'll formalize raw-data references with `source()`, load small CSVs with `dbt seed`, and add `not_null` / `unique` tests.
- **Session 4 onward** — Modeling, Jinja, materializations, snapshots, governance, CI/CD.

Keep this handout nearby — we will come back to the files and the diagram repeatedly throughout the rest of the course.